

Week3

Task1

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

Source code:

```
#include <stdio.h>
#include <string.h>
#define MAX 1000
#define MAX_STRINGS 100

int main() {
    char input[MAX];
    char quoted[MAX_STRINGS][MAX];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    int inSingleQuote = 0;
    int j = 0;
    char temp[MAX];

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\'' && !inSingleQuote) {
            inSingleQuote = 1;
            j = 0;
            continue;
        }

        if (input[i] == '\'' && inSingleQuote) {
            inSingleQuote = 0;

            temp[j] = '\0';

            if (count < MAX_STRINGS) {
                strcpy(quoted[count], temp);
                count++;
            }

            continue;
        }

        if (inSingleQuote) {
```

```

        if (inSingleQuote) {
            if (j < MAX - 1) {
                temp[j++] = input[i];
            }
        }
    }

    if (inSingleQuote) {
        printf("Error: Unmatched single quote.\n");
        return 1;
    }

    printf("\nQuoted strings:\n");

    if (count == 0) {
        printf("No single-quoted strings found.\n");
    } else {
        for (int i = 0; i < count; i++) {
            printf("%d: '%s'\n", i + 1, quoted[i]);
        }
    }

    return 0;
}

```

Output:

```

Enter command: echo 'Hello $USER' 'This is literal text'

Quoted strings:
1: 'Hello $USER'
2: 'This is literal text'

```

Task2:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Source Code:

```

#include <stdio.h>
#include <string.h>

#define MAX 1000
#define MAX_STRINGS 100

int main() {
    char input[MAX];
    char quoted[MAX_STRINGS][MAX];
    int count = 0;
    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    int inSingleQuote = 0;
    int j = 0;
    char temp[MAX];
    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' && !inSingleQuote) {
            inSingleQuote = 1;
            j = 0;
            continue;
        }
        if (input[i] == '"' && inSingleQuote) {
            inSingleQuote = 0;
            temp[j] = '\0';
            if (count < MAX_STRINGS) {
                strcpy(quoted[count], temp);
                count++;
            }
            continue;
        }
        if (inSingleQuote) {
            if (j < MAX - 1) {
                temp[j++] = input[i];
            }
        }
    }
    if (inSingleQuote) {
        printf("Error: Unmatched single quote.\n");
        return 1;
    }
}

```

```

    }
    if (inSingleQuote) {
        printf("Error: Unmatched single quote.\n");
        return 1;
    }
    printf("\nQuoted strings:\n");
    if (count == 0) {
        printf("No single-quoted strings found.\n");
    } else {
        for (int i = 0; i < count; i++) {
            printf("%d: '%s'\n", i + 1, quoted[i]);
        }
    }
    return 0;
}

```

Output:

```
root@ubuntu:~# echo 'Hello $USER' 'This is literal text'
Enter command: echo 'Hello $USER' 'This is literal text'

Quoted strings:
1: 'Hello $USER'
2: 'This is literal text'
```

Task3:

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

Source Code:

```
#include <stdio.h>
#include <string.h>
#define MAX 1000
int main() {
    char input[MAX];
    char output[MAX];
    int i = 0, j = 0;

    printf("Enter input: ");
    fgets(input, sizeof(input), stdin);
    while (input[i] != '\0' && input[i] != '\n') {

        if (input[i] == '\\') {
            i++;

            if (input[i] == '\\0' || input[i] == '\\n')
                break;

            switch (input[i]) {
                case 'n':
                    output[j++] = '\\n';
                    break;

                case 't':
                    output[j++] = '\\t';
                    break;

                case 'r':
                    output[j++] = '\\r';
                    break;

                case 'b':
                    output[j++] = '\\b';
                    break;

                case ' ':
                    output[j++] = ' ';
                    break;

                case '\\':
```

```
        case '\\':
            output[j++] = '\\';
            break;

        case '\':
            output[j++] = '\\';
            break;

        case '"':
            output[j++] = '"';
            break;

        case '$':
            output[j++] = '$';
            break;

        case '&':
            output[j++] = '&';
            break;

        case '|':
            output[j++] = '|';
            break;

        case ';':
            output[j++] = ';';
            break;

        default:
            output[j++] = '\\';
            output[j++] = input[i];
            break;
    }
}
else {
    output[j++] = input[i];
}

i++;
```

```

        output[j++] = '&';
        break;

    case '|':
        output[j++] = '|';
        break;

    case ';':
        output[j++] = ';';
        break;

    default:
        output[j++] = '\\';
        output[j++] = input[i];
        break;
    }
}
else {
    output[j++] = input[i];
}

i++;
}

output[j] = '\\0';

printf("\nParsed Output:\n");
printf("%s\n", output);

return 0;
}

```

Output:

```

Enter input: Hello\ World

Parsed Output:
Hello World

```

Task4

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    pid_t pid;
    int status;

    if (argc < 2)
    {
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execvp(argv[1], &argv[1]);

        perror("execvp failed");
        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
}
```

```

        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid failed");
            return 1;
        }

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status))
        {
            printf("Child terminated by signal: %d\n",
                WTERMSIG(status));
        }
    }

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./sung ls
Parent Process
Parent PID: 677
Child Process
Created Child PID: 678
Child PID: 678
Parent PID: 677
OS      bc.c      cd.c      co      codes.c      diksh      fil.c      gm      jaf.c      operating  programming
sample.txt  source.txt  sung.c.save
OSSP     c.c.save  child     co.c      d.c          diksh.c    files    gm.c      nan        os          programming.c
ea       source_code  task2
OS_Lab   cc        child.c  code      dance        dog.c      files.c   hello    nan.c      os.c        programming.c.save
ea.c     source_code.c task2.c
b.c.save cc.c      clear    code.c    dance.c      echo       fork      hello.c   nano.543.save osc.c       programs
ee       sung      trace.txt
bb.c.save cd        clear.c  codes     destination.txt fil        fork.c    jaf       oc.c       program.c   programs.c
ee.c     sung.c
Child exited with status: 0

```